CAHIER DES CHARGES

Agent de Voyage IA — DocumentDB + MCP

Application Flask + Agent IA + Serveur MCP DocumentDB + cache vectoriel

Type de document : Cahier des charges fonctionnel et technique

Date : 22 septembre 2026

Référence : emmanueldeletang/Documentdb_travel_mcp_sample (données et architecture)

Socle skills : Azure/documentdb-agent-kit

1. Contexte et objectifs

1.1 Contexte

Ce cahier des charges décrit une application « agent de voyage » qui permet d'interroger en langage naturel une base de données de voyages/réservations (destinations, vols, clients, réservations) hébergée sur Azure DocumentDB (compatible MongoDB). Il s'appuie directement sur l'exemple public emmanueldeletang/Documentdb_travel_mcp_sample, qui démontre l'utilisation du serveur MCP officiel microsoft/documentdb-mcp couplé aux skills d'Azure/documentdb-agent-kit.

Contrairement à une architecture où l'on développerait un serveur MCP « métier » sur mesure (outils dédiés recherche de vols, comparaison, etc.), l'approche retenue ici réutilise le serveur MCP générique DocumentDB : l'agent dispose d'outils génériques (recherche, comptage, agrégation, gestion des index) et s'appuie sur les skills du kit pour produire de bonnes requêtes MongoDB et de bons index, plutôt que sur des outils métier codés à la main.

L'application Flask ajoute une couche de valeur au-dessus de ce socle : une interface de chat, et surtout un cache de questions/réponses avec similarité vectorielle, qui réduit le coût et la latence des appels au LLM pour les questions identiques ou proches.

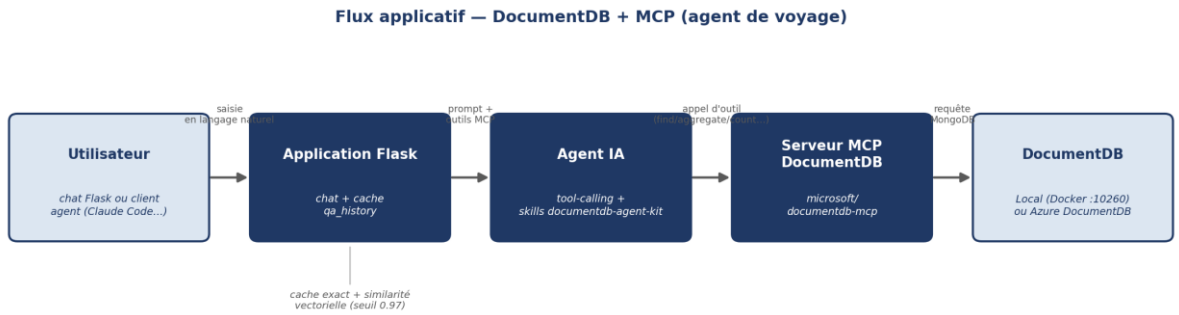
1.2 Objectifs du projet

#	Objectif
O1	Permettre d'interroger en langage naturel une base de voyages (destinations, vols, clients, réservations) sans écrire de requête MongoDB
O2	Réutiliser le serveur MCP officiel microsoft/documentdb-mcp plutôt que redévelopper des outils métier, pour bénéficier de ses outils génériques (find, aggregate, count, index...)
O3	Mobiliser les skills d'Azure/documentdb-agent-kit pour guider l'agent vers de bonnes requêtes, agrégations et stratégies d'indexation (règle ESR, etc.)
O4	Fournir une interface web Flask avec chatbot, restituant à la fois la réponse en langage naturel et, sur demande, la requête MongoDB générée
O5	Réduire le coût et la latence des questions répétées via un cache exact + un cache par similarité vectorielle (collection qa_history)
O6	Permettre un fonctionnement aussi bien en local (DocumentDB Local via Docker) qu'en production (Azure DocumentDB), par simple changement de chaîne de connexion

1.3 Hors périmètre (v1)

- Paiement / réservation réelle de billets (les réservations de la base sont des données de démonstration)
- Intégration à de vraies API de compagnies aériennes (GDS type Amadeus/Sabre) — les données sont synthétiques (seed aléatoire), à usage de démonstration uniquement
- Authentification multi-utilisateurs avancée (SSO, rôles fins) côté Flask
- Recherche plein texte (BM25) et recherche vectorielle sur les descriptions de destinations — identifiées comme évolutions possibles via les skills full-text-search / vector-search du kit, non couvertes en v1

2. Vue d'ensemble de l'architecture



L'utilisateur pose sa question en langage naturel, via l'interface de chat Flask (ou directement via un client agent tel que Claude Code, Cursor ou GitHub Copilot). L'agent IA, guidé par les skills du kit DocumentDB, choisit le bon outil du serveur MCP DocumentDB (recherche, comptage, agrégation, optimisation d'index...) et l'exécute contre la base DocumentDB — locale en développement, Azure DocumentDB en production. Côté Flask, une couche de cache (collection qa_history) court-circuite l'appel au LLM lorsque la question a déjà été posée, à l'identique ou de façon très similaire.

2.1 Principe de fonctionnement

1. L'utilisateur saisit une question en langage naturel (ex. « Top 3 des villes de destination par chiffre d'affaires »).

2. Si l'application Flask est utilisée : elle calcule l'empreinte (embedding) de la question et vérifie d'abord dans qa_history s'il existe une correspondance exacte, puis une correspondance par similarité vectorielle (seuil configurable, 0.97 par défaut).

3. En cas de cache miss, la question est transmise à l'agent IA avec les outils exposés par le serveur MCP DocumentDB.

4. L'agent choisit l'outil adapté (find_documents, count_documents, aggregate, optimize_find_query, etc.), guidé par les skills du kit (indexation ESR, modélisation, etc.).

5. Le serveur MCP exécute la requête MongoDB correspondante contre la base traveldb et retourne un résultat structuré.

6. L'agent formule la réponse en langage naturel ; Flask l'affiche et enregistre la question, la réponse, l'embedding et les métriques (tokens, temps de réponse) dans qa_history pour les prochaines requêtes similaires.

3. Modèle de données MongoDB (base « traveldb »)

La base de données traveldb reprend telle quelle la structure du jeu de données de démonstration du dépôt de référence : 4 collections métier, alimentées par le script script/load-data.py, plus une collection technique ajoutée par l'application Flask.

3.1 Collections et volumétrie de référence

Collection	Volume (démo)	Rôle
destinations	10 documents	Référentiel des destinations (ville, pays, région, catégorie, note)
flights	10 documents	Référentiel des vols proposés (compagnie, tarif de base)
customers	12 documents	Clients (nom, niveau de fidélité)
reservations	24 documents	Réservations reliant client, destination et vol (statut, dates, prix, paiement)

Collection	Volume (démo)	Rôle
qa_history	créée par flask-app	Historique des questions/réponses du chatbot (cache exact + vectoriel)

3.2 Schéma indicatif des champs (déduit des requêtes types)

Collection	Champs principaux
destinations	_id (destination_id), city, country, region, category, rating
flights	_id (flight_id), airline, base_fare
customers	_id (customer_id), customer_name, loyalty_tier (platinum / gold / …)
reservations	customer_id, customer_name, destination_id, destination_city, flight_id, status (confirmed / cancelled / completed), check_in, nights, total_price, payment.method, payment.paid
qa_history	question, question_normalized, embedding (vecteur), answer, tokens_used, tokens_saved, response_time_ms, created_at

Remarque : les dates (ex. check_in) sont stockées sous forme de chaînes « AAAA-MM-JJ », ce qui permet des comparaisons de plage par simple comparaison de chaînes.

3.3 Relations entre collections

- reservations → customers via customer_id
- reservations → destinations via destination_id (et destination_city dénormalisé pour éviter des $lookup sur les requêtes simples)
- reservations → flights via flight_id

3.4 Chargement des données

Le script script/load-data.py importe les fichiers JSON du dossier data/ (destinations, flights, customers, reservations) dans traveldb et crée les index nécessaires. Il doit afficher en sortie le décompte attendu par collection (10 / 10 / 12 / 24) pour valider le chargement.

3.5 Indexation recommandée

- reservations : index composé { status: 1, check_in: 1 } pour les filtres statut + date (règle ESR du kit)
- reservations : index sur destination_city et sur customer_id pour les agrégations fréquentes
- destinations : index sur region et category pour les requêtes de type « destinations d'Asie notées > 4.5 »
- Toute création d'index doit être vérifiée avec optimize_find_query / explain_aggregate_query avant/après (absence de COLLSCAN)

4. Spécifications du serveur MCP

4.1 Composant réutilisé

Le projet ne développe pas de serveur MCP métier : il réutilise le serveur MCP officiel microsoft/documentdb-mcp, lancé via npx -y github:microsoft/documentdb-mcp (le serveur n'étant pas publié sur npm, l'ancien nom de paquet documentdb-mcp-server ne fonctionne plus). L'agent y accède au travers des skills du kit Azure/documentdb-agent-kit, qui l'orientent vers les bons outils et les bonnes pratiques MongoDB/DocumentDB.

4.2 Outils génériques exposés (utilisés par le sample)

Outil	Usage typique dans l'agent de voyage
list_databases	Vérifier la présence et l'accès à traveldb
sample_documents	Afficher des exemples de documents (ex. « montre 5 réservations »)
find_documents	Recherches filtrées (ex. réservations confirmées vers Tokyo, triées par date)
count_documents	Comptages rapides (ex. réservations confirmées vs annulées)
aggregate	Agrégations analytiques (chiffre d'affaires, top destinations, dépense moyenne par client, revenu par mois...)
optimize_find_query / explain_aggregate_query	Diagnostic de requêtes lentes (détection de COLLSCAN)
create_index	Création d'index recommandés (ex. { status: 1, check_in: 1 })
list_indexes / index_stats	Inspection des index existants et de leur taille

4.3 Configuration et profils de connexion

La chaîne de connexion n'est jamais fournie par l'agent : elle est définie côté configuration MCP (variable DOCUMENTDB_URI) via un profil de connexion nommé ("default"). Exemple de configuration en transport stdio local :

```
"TRANSPORT": "stdio",
"AUTH_REQUIRED": "false",
"TRUST_LOCAL_STDIO": "true",
"CONNECTION_PROFILES":
"{\"default\":{\"authMode\":\"connectionString\",\"uriEnv\":\"DOCUMENTDB_URI\",\"tier\":\"write\"}}",
"DOCUMENTDB_URI": "mongodb://user:pass@host:10260/?tls=true&tlsAllowInvalidCertificates=true",
"ENABLE_WRITE_TOOLS": "true",
"ENABLE_MANAGEMENT_TOOLS": "true"
```

Pour un agent strictement en lecture seule, ENABLE_WRITE_TOOLS et ENABLE_MANAGEMENT_TOOLS doivent rester à false (valeur par défaut) : l'agent de voyage n'a besoin d'aucun de ces deux flags pour répondre aux questions analytiques, ils ne sont utiles que pour la création d'index ou la maintenance.

4.4 Bascule local ↔ Azure DocumentDB

Le changement d'environnement se limite à la valeur de DOCUMENTDB_URI : chaîne locale (mongodb://…@localhost:10260/…) en développement, ou chaîne du cluster Azure DocumentDB (mongodb+srv://…mongocluster.cosmos.azure.com/?tls=true&authMechanism=SCRAM-SHA-256) en production, obtenue depuis le portail Azure (Paramètres → Chaînes de connexion). Le profil de connexion peut rester nommé "default".

4.5 Exigences de sécurité

- Aucune chaîne de connexion transmise ou modifiable par l'agent à l'exécution
- Outils d'écriture et d'administration désactivés par défaut (lecture seule) ; activation explicite si des index doivent être créés par l'agent
- En local, TLS avec certificat auto-signé (tlsAllowInvalidCertificates=true) — à ne jamais utiliser tel quel en production
- En production, connexion à un cluster Azure DocumentDB avec certificat valide et authentification SCRAM-SHA-256 ou Microsoft Entra ID

5. Spécifications de l'application Flask

5.1 Fonctionnalités

- Interface de chat : saisie de la question, affichage de la réponse, et sur demande de la requête MongoDB générée par l'agent
- Endpoint principal POST /api/ask : reçoit la question, retourne la réponse ainsi que les métadonnées de traçabilité (cache hit/miss, tokens consommés, tokens économisés, temps d'exécution en ms)
- Gestion de session pour le contexte conversationnel (ex. « décompose maintenant ce chiffre d'affaires par niveau de fidélité »)

5.2 Couche de cache (mise en cache + recherche vectorielle)

L'application Flask ajoute une couche de persistance pour réduire le coût récurrent lié au LLM, matérialisée par la collection qa_history :

1. L'utilisateur pose une question.

2. L'application calcule l'empreinte vectorielle (embedding) de la question, si un modèle d'embedding est configuré.

3. Elle vérifie qa_history : d'abord une correspondance exacte sur la question normalisée, puis une correspondance par similarité vectorielle (seuil CACHE_SIMILARITY_THRESHOLD, 0.97 par défaut).

4. En cas de succès (cache hit) : la réponse et la trace sont retournées immédiatement, sans nouvel appel au LLM ni au serveur MCP.

5. En cas d'échec (cache miss) : l'agent exécute le tool-calling MCP habituel, puis le résultat est enregistré dans qa_history pour les prochaines questions identiques ou proches.

5.3 Architecture applicative (arborescence indicative, alignée sur le dépôt de référence)

```
agent/                # logique d'orchestration de l'agent (tool-calling MCP)
data/                 # jeux de données JSON de démonstration
  destinations.json
  flights.json
  customers.json
  reservations.json
flask-app/            # application web (chat + cache qa_history)
  app.py / routes/    # ex. route /api/ask
  requirements.txt    # inclut pymongo[srv]
script/
  load-data.py        # importe data/*.json et crée les index
docker-compose.yml    # DocumentDB Local (port 10260)
.env.example          # identifiants + chaîne de connexion
mcp.json              # configuration du serveur MCP DocumentDB
NL-QUERIES.md         # catalogue de questions en langage naturel
README.md
```

5.4 Technologies

- Backend : Python 3.10+, Flask
- Driver MongoDB : pymongo[srv]
- Base de données : DocumentDB Local (Docker, image ghcr.io/microsoft/documentdb/documentdb-local, port 10260) en développement ; Azure DocumentDB en production
- Serveur MCP : microsoft/documentdb-mcp, lancé via npx (Node.js 20+ requis)
- Modèle d'embedding (optionnel) : configuré via AZURE_OPENAI_EMBEDDING_DEPLOYMENT ou OPENAI_EMBEDDING_MODEL, nécessaire pour activer le cache par similarité vectorielle

6. Spécifications du chatbot / agent IA

6.1 Comportement attendu

- Traduire une question en langage naturel en appel d'outil MCP adapté, sans jamais écrire lui-même la requête MongoDB à la place de l'outil
- S'appuyer sur les skills d'Azure/documentdb-agent-kit pour choisir la bonne syntaxe MongoDB/agrégation et pour recommander des index pertinents
- Nommer explicitement la collection cible si le contexte est ambigu (ex. « dans la collection reservations »)
- Pouvoir restituer, sur demande, la requête MongoDB générée (traçabilité), comme illustré dans NL-QUERIES.md
- Conserver le contexte conversationnel pour enchaîner des demandes de raffinement (ex. « décompose maintenant par niveau de fidélité »)

6.2 Exemples de questions en langage naturel couvertes (catalogue de référence)

Catégorie	Exemple de question	Outil MCP principal
Consultation	« Montre-moi 5 réservations. »	sample_documents
Consultation filtrée	« Liste les réservations confirmées vers Tokyo, check-in le plus récent d'abord. »	find_documents
Comptage	« Combien de réservations sont confirmées vs annulées ? »	count_documents
Agrégation	« Top 3 des villes de destination par chiffre d'affaires. »	aggregate
Agrégation avec jointure	« Nombre moyen de nuits par région. »	aggregate ($lookup)
Performance	« Cette requête est lente : réservations filtrées par statut et date de check-in. Comment l'indexer ? »	optimize_find_query / create_index
Index	« Montre les index de la collection reservations et leur taille. »	list_indexes / index_stats

6.3 Évolutions couvertes par d'autres skills du kit (hors périmètre v1)

- Recherche plein texte (BM25) sur les tags de destinations → skill full-text-search
- Recherche sémantique sur les descriptions de destinations → skill vector-search (cosmosSearch, DiskANN/HNSW)
- Choix embed vs référence pour les réservations → skill data-modeling
- Réglage du pool de connexions pour une charge serverless → skill connection

7. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Une question déjà posée (cache hit exact) doit répondre quasi instantanément, sans nouvel appel LLM/MCP
Coût	Le cache par similarité vectorielle doit réduire mesurablement les tokens consommés sur les questions reformulées (traçé via tokens_saved)
Sécurité	Chaîne de connexion DocumentDB jamais exposée côté client ; secrets uniquement en .env, non versionnés

Catégorie	Exigence
Fiabilité	Le chargement des données (load-data.py) doit produire les décomptes attendus (10/10/12/24) avant toute mise en service
Portabilité	Bascule locale/Azure limitée à la variable DOCUMENTDB_URI, sans changement de code
Traçabilité	Chaque réponse /api/ask doit exposer cache hit/miss, tokens utilisés/économisés et temps d'exécution
Observabilité	Diagnostic des requêtes lentes possible via optimize_find_query / explain_aggregate_query avant mise en production d'un nouvel usage

8. Cas d'usage et jeux de données

Les cas d'usage ci-dessous reprennent le catalogue NL-QUERIES.md du dépôt de référence, qui sert de jeu de tests de recette fonctionnelle pour l'agent.

Cas d'usage	Description
Chiffre d'affaires global	« Quel est notre chiffre d'affaires total réservé (confirmé + terminé) ? »
Top destinations	« Top 3 des villes de destination par chiffre d'affaires. »
Segmentation clients	« Quels clients platinum ou gold ont un voyage vers Tokyo ? »
Analyse temporelle	« Chiffre d'affaires par mois pour 2026. »
Analyse par moyen de paiement	« Quels moyens de paiement sont les plus utilisés, et pour quel chiffre d'affaires ? »
Diagnostic de performance	« Cette requête est lente — comment indexer les réservations filtrées par statut et date de check-in ? »
Absence de résultat / ambiguïté	L'agent précise la collection ou élargit les critères si aucun résultat ou si plusieurs collections sont plausibles

8.1 Conseils d'usage identifiés dans le dépôt de référence

- Nommer la collection si l'agent se trompe de cible : « … dans la collection reservations »
- Demander explicitement la requête générée : « … et montre-moi l'agrégation MongoDB utilisée »
- Enchaîner les questions pour affiner une analyse : l'agent conserve le contexte de la conversation

9. Livrables attendus

- docker-compose.yml lançant DocumentDB Local (port 10260)
- .env.example listant les identifiants et la chaîne de connexion
- mcp.json configurant le serveur MCP DocumentDB (microsoft/documentdb-mcp) pour les clients supportés (Claude Code, Cursor, VS Code/Copilot, Gemini CLI)
- Jeux de données data/*.json (destinations, vols, clients, réservations) et script script/load-data.py
- Application flask-app/ avec chat, endpoint /api/ask et couche de cache qa_history
- Documentation README.md (installation en 5 étapes) et NL-QUERIES.md (catalogue de questions de recette)
- Configuration des skills Azure/documentdb-agent-kit dans l'environnement de l'agent (npx skills add Azure/documentdb-agent-kit)

10. Planning indicatif

Phase	Contenu	Durée indicative
1. Socle DocumentDB	docker-compose, .env, chargement des données de démonstration, vérification des décomptes	2 jours
2. Intégration MCP	Configuration mcp.json, test des outils génériques (list_databases, sample_documents, find_documents)	2 jours
3. Skills & requêtes	Installation des skills du kit, validation du catalogue NL-QUERIES.md (comptages, agrégations, index)	3 jours
4. Application Flask	Chat, endpoint /api/ask, affichage de la requête générée	3-4 jours
5. Couche de cache	Collection qa_history, cache exact, embeddings, similarité vectorielle, métriques de tokens	3-4 jours
6. Recette & bascule Azure	Tests sur le catalogue de questions, bascule vers un cluster Azure DocumentDB	2-3 jours

11. Critères d'acceptation

☐ Le chargement des données produit les décomptes attendus : destinations (10), flights (10), customers (12), reservations (24).

☐ L'agent répond correctement aux questions du catalogue NL-QUERIES.md en utilisant l'outil MCP approprié (find_documents, count_documents, aggregate...).

☐ Sur demande, l'agent restitue la requête MongoDB ou l'agrégation qu'il a générée.

☐ Une question déjà posée à l'identique obtient une réponse en cache (cache hit) sans nouvel appel LLM.

☐ Une question reformulée proche d'une question existante déclenche un cache hit par similarité vectorielle (seuil configurable).

☐ Chaque réponse /api/ask expose cache hit/miss, tokens utilisés/économisés et temps d'exécution.

☐ Les outils d'écriture/administration MCP restent désactivés par défaut (ENABLE_WRITE_TOOLS=false).

☐ Le passage de DocumentDB Local à Azure DocumentDB ne nécessite que la modification de DOCUMENTDB_URI.

☐ Une requête lente identifiée par l'agent (COLLSCAN) se voit proposer un index conforme à la règle ESR du kit.

12. Risques identifiés

Risque	Impact	Mitigation
Le serveur MCP n'est pas publié sur npm (dépendance à github:microsoft/documentdb-mcp)	Échec de lancement si le nom de paquet documentdb-mcp-server est utilisé	Utiliser exclusivement npx -y github:microsoft/documentdb-mcp, documenté dans le README
Cache vectoriel mal configuré (pas de modèle d'embedding)	Aucun cache hit par similarité, coût LLM non réduit	Configurer explicitement AZURE_OPENAI_EMBEDDING_DEPLOYMENT ou OPENAI_EMBEDDING_MODEL et valider avec des questions reformulées

Risque	Impact	Mitigation
Données de démonstration synthétiques	Résultats peu représentatifs d'un usage réel	Cadrer explicitement l'usage en v1 comme démonstration ; prévoir un plan de migration de données réelles pour une v2
Agent mal configuré en écriture (ENABLE_WRITE_TOOLS=true) en production	Risque de modification/suppression accidentelle de données	Garder ces flags à false hors tâches d'administration explicites, et les réactiver ponctuellement seulement
TLS auto-signé recopié tel quel en production	Connexion non sécurisée sur un cluster réel	Retirer tlsAllowInvalidCertificates lors du passage à Azure DocumentDB et utiliser SCRAM-SHA-256 ou Entra ID

Ce cahier des charges reprend fidèlement l'architecture, le modèle de données et le catalogue de questions du dépôt public emmanueldeletang/Documentdb_travel_mcp_sample (consulté le 22 septembre 2026), lui-même bâti sur les skills et le serveur MCP d'Azure/documentdb-agent-kit. Toute évolution du dépôt source (nouvelles collections, nouveaux outils MCP) devra être répercutée dans une révision ultérieure de ce document.